Threat Model

Operation fsoc: Case Study

Gurmann Basran

Revised 2026-06-15 (rev. 3).

1 What this document is

This is the threat model I wrote before I touched the first firewall rule of the security hardening phase. It is the document I return to every time I add a new service, redraw a trust boundary, or make any change I cannot cleanly reverse. What follows is the category-level version of it; specific tool names, service names, exposed port counts, upstream provider names, and ISP identity have been deliberately stripped. The exercise itself is what I want to show, not the particular surface of my network.

I am publishing this because writing a threat model was genuinely the most useful half-day of the project. Deciding what I am defending against forces me to decide what I am explicitly *not* defending against, and that decision in turn tells me which phase each piece of attack surface belongs to. This is the third revision, and it keeps folding in changes as they ship; where a row used to say a control was deferred and the control has since landed, the row now says so. The largest addition this time is a row of defense the first two versions did not have at all, because the failure it covers has nothing to do with an attacker and everything to do with the system falling over under its own weight.

2 What the model defends against

The classes of threat I am designing around:

- External reconnaissance and exploitation of publicly-reachable services. The perimeter exposes a small, carefully-chosen set of inbound paths, and every one of those paths is attack surface.
- Lateral movement between trust zones. If one of my devices in the home zone gets compromised (malware, drive-by, whatever), the attacker should not have a direct path into the services zone, the anonymity zone, or the offensive-learning zone. They should have to come through the same gate a legitimate request would.
- A stolen device with valid credentials escalating to broad network access. Any device I carry (phone, admin workstation, portable USB) has some credentials on it. If any of them is stolen, the stolen credentials should grant a narrow slice of access, not free run of the network. Fleet access now rests entirely on short-lived signed certificates rather than long-lived keys; the cutover is complete, the last long-lived static key has been removed, and a self-checking gate refuses to let one creep back in. A credential lifted from a device ages out on its own, and revoking it is a single central action.

- Credential theft cascading across services. A compromised cookie on one internal app should not mean compromise of every internal app; the SSO layer scope has to be right. An exfiltrated API token should be scope-locked to the minimum it needs.
- Passive surveillance of the queries leaving the network. The resolution path for every name lookup in the network is chained so that plaintext queries never leave the site.
- Supply-chain compromise reaching the fleet through a package, an image, or an update. Public registries have been poisoned more than once in the last year. Updates are reviewed rather than applied unattended, and when a campaign breaks I run a standing forensic sweep that checks every host against the published indicators and turns any confirmed match into a perimeter block.
- Resource-exhaustion failure coming from inside (a runaway process, unbounded log or container growth, a job that eats a host's memory) or outside (flooding the perimeter). This class earned a dedicated defense only after it bit me: a capacity watchdog now runs from an independent host and watches for sustained pressure building, rather than waiting for the crash to announce itself. The lesson behind that addition is in the methodology and lessons documents.
- Single-host log tampering that hides post-compromise activity. Centralized off-host log forwarding is the next detection milestone and its research is done; until it lands, the mitigations that do not require new infrastructure carry the load, and a live per-host and per-container egress monitor flags traffic that should not be leaving.

3 What the model explicitly does not defend against

A threat model with no exclusions is a fantasy. These are the risks I have decided not to address yet, and each of them is assigned either to a later phase or to the residual-risk column. The point of naming them is so I do not accidentally trick myself into thinking the system is hardened against something it is not.

- **Physical access to the hardware.** Boot from external media, cold-boot attack against memory, drive extraction. This class is deferred to the physical-security phase, which adds BIOS-password hardening, drive-level encryption, and tamper-evidence. The interim mitigation is that the hardware lives in my apartment and I lock my door.
- **Kernel-level container escapes against a zero-day.** The detection layer will catch the post-compromise activity if the signature is known; preventing the class entirely is not a realistic goal for a solo-operator project.
- **ISP-level traffic analysis.** Even with encrypted name resolution, the ISP can still see I am talking to a given edge CDN. The egress-VPN phase collapses the visible set of destinations down to one encrypted tunnel, which is the mitigation; until that phase lands, the ISP still sees my destination patterns.
- **Nation-state legal process against upstream providers.** Domain registrars, DNS providers, and cloud-backup services all keep records. If a state-level actor compels them, the records are

visible. The mitigation is encryption-at-rest for anything personal plus the acceptance that this is residual risk for any system hosted on rented infrastructure anywhere.

- **Total power loss, fire, or theft of the hardware.** Partial mitigation through off-host encrypted backup; full geographic redundancy is out of scope for a student-budget project.
- **Admin-workstation compromise.** Browser RCE, operating-system malware, anything that owns the machine I use for administrative access. This is the residual risk the most recent milestone set out to narrow, and the companion document on the workstation migration covers it in full. The mitigations now in place: the workstation holds a short-lived signed certificate from a private certificate authority rather than long-lived per-host keys, so a compromise is bounded by the certificate's lifetime and revoked centrally; the actual secrets stay in the vault behind a second factor and never land on the machine; and the workstation itself was rebuilt to the same hardened standard as the infrastructure rather than being a soft consumer box with administrative reach. Full mitigation would still require either an air-gapped admin path or a hardware second-factor touched on every privileged action; I have neither yet, so a live compromise of the machine still owns the session while it lasts.

4 Assume-breach scenarios

The part of the threat model that actually drives design decisions is the assume-breach table. For every trust boundary I care about, I wrote down what happens if it gets crossed: the blast-radius limit, and how I would detect the event. A new component does not get deployed until I can fill in both columns for it.

The rows below are described at the abstraction level that matters for the design discipline, with the specific services and protocols generalized.

Scenario	Blast-radius limit	Detection signal
A remote-access credential is extracted from a device I own.	The credential is scoped narrowly at the perimeter: only the services that device actually needs, nothing else. It is a short-lived signed certificate, so it ages out on its own, and revoking it is a single central action.	SSO anomaly signal (failed second-factor, or successful connect from an unexpected location), plus a real-time push alert.
A device inside the home zone is compromised.	The inter-zone firewall policy blocks direct access to the services zone. Any request from the home zone has to come through the same gate a public request would: SSO with second-factor.	Behavioral-defense signatures on rate-limit and scan patterns, plus canary tokens that are deployed and drill-tested across file, DNS, and credential vectors; tripping any one of them pages me.

A container on the services host is popped.	Capability drops, isolated container networking, and per-service egress controls mean the attacker cannot reach the home, anonymity, or offensive zones directly. Host-level detection is watching while they try.	A live per-container egress monitor flags any unexpected outbound destination now; centralized off-host log forwarding is the next detection milestone.
The secrets vault master credential is exfiltrated.	Second-factor still gates access to the vault. The vault is not reachable from the WAN. Daily encrypted backups with a rotating encryption recipient mean an attacker cannot quietly delete the contents without detection.	SSO audit log entries on abnormal vault access.
An API credential used by one of the internal services is exfiltrated.	The credential is scope-locked to exactly what the service needs (the minimum verb on the minimum resource). Upstream audit alerts fire on any zone or account change. Rotation is a one-command action.	Upstream provider's audit log, plus the watchdog that re-reads the credential's scope periodically.
The firewall itself is destroyed.	Off-host encrypted config backup is kept current; a restore completes in about 15 minutes. An emergency-management IP on a separate physical NIC lets me reach the hypervisor without routing through the firewall.	An out-of-band watchdog running on a different physical host alerts after a short window of unreachability.
The admin workstation is compromised.	It carries a short-lived signed certificate, not the keys to the kingdom; the secrets live in the vault behind a second factor, not on the disk. The blast radius is the certificate's remaining lifetime, after which the access ages out, and revocation is central and immediate.	Real-time push on every privileged login across the fleet, with the user, source, and a location lookup; a login I did not start is a page-one event.

A host exhausts its own resources: a runaway process, a memory leak, a disk or scratch volume filling up.	Per-host isolation keeps one host's exhaustion from cascading to the others. A capacity watchdog that runs from a host which does not depend on the one being watched catches sustained pressure while there is still time to intervene, rather than after the host has already fallen over.	Operator-tuned thresholds on sustained processor saturation, memory and swap pressure, and filesystem fill, each pushed as a real-time alert rather than discovered in a morning log review.
The primary backup medium is lost, or backups silently stop being produced.	Backups are written off-host and encrypted, and the encryption key is escrowed off the fleet, so the loss of any single location is survivable. The archives have been proven to decrypt and restore end to end, not merely to exist on disk.	A staleness alarm fires if a fresh backup does not appear inside its expected window, so a backup that quietly stops is itself a page-able event rather than a surprise discovered during a recovery.

The pattern is what matters: every row pairs a containment bound with a detection. I do not accept a row that says "the attacker would be in a lot of places." I do not accept a row that says "we would figure it out eventually." If I cannot fill in both columns, the component does not deploy.

5 Severity categories

The first audit that came out of this threat model used four severity categories. These are my working definitions; they are not CVSS and I am not pretending they are. They are good enough for an individual operator to prioritize a backlog. The later, larger audit moved to tagging findings by the action each one demanded instead, which the methodology document explains; the severity definitions below are still how I size a single issue in my head.

- **Critical.** An attacker on the internet, or on a compromised adjacent host, could reach full compromise in minutes. Examples in this audit included default authentication modes on a service that should have had them disabled, an administrative surface reachable without encryption, and a credential whose scope was broader than anything actually using it required.

- **High.** A breach of one layer would pivot or escalate with no friction. Examples included a trust zone with an allow-all rule, a service-level policy that passed all traffic where it should have passed a narrow list, and cluster-management services trusting the wrong set of hosts.

- **Medium.** Violations of least privilege or surface-exposed state that are not directly exploitable today but should not be left lying around. Stale configuration referencing retired addresses,

overly broad bind addresses on internal services, unencrypted service-to-service traffic where encryption was practical, container supervisors holding privileges they did not actually need.

- **Low.** Defense-in-depth items whose mitigation cost is low enough to land anyway. Amplification vectors, default admin credentials on hardware that is already behind the perimeter, service banners leaking version strings, legacy accept rules for retired subnets.

There is also a *red-team additional* category I ended up creating for findings that came out of a second, more adversarial review a week after the first pass. Those tended to be the interesting ones. The first pass catches the obvious configuration gaps; the second pass catches the things that require a little more creative thinking.

6 How the model is kept current

The threat model is not a one-time document. I re-read it at the start of every new phase, before I touch anything. If the phase adds a new service, a new trust zone, or a new boundary, the relevant sections get updated in the same commit that adds the component. The assume-breach table is gate-checked on every deploy: a component that cannot name its blast-radius limit and its detection signal does not get deployed. This is the third revision of the public version. The controls that moved from in-progress to fully shipped since the last one, the completed cutover to certificate-only fleet authentication, the off-fleet escrow of the backup encryption key, and the staleness alarm on the backups themselves, are exactly the rows that prove the model is maintained rather than written once and abandoned. The newest row, the one for a host exhausting its own resources, is there because the model is supposed to grow a row every time reality teaches me a failure mode I had not written down, and this revision it taught me one.

The full findings table, which lives in the private planning directory rather than in this public writeup, is linked by ID to the commit that closed each finding. There is no “fix it later” entry without a phase assignment. The paper trail is the point; months from now, when I am asking myself why a particular rule exists, the commit message and the finding ID should give me the answer without having to rebuild the reasoning from scratch.